

WIPRO SELENIUM – MODULE 1 LAB WORKBOOK

Automation with Selenium

Student Name: Debasmita Majhi

Roll Number: 12023002029034

Course: Selenium Automation

Module: Module 1 – Automation with Selenium

Technology: Python + Selenium WebDriver

Browser: Google Chrome

IDE: Visual Studio Code

Experiment 1.1 — Introduction and Different Flavors of Selenium

Problem Statement

To understand the basic working of Selenium WebDriver by launching a Chrome browser, opening a website, and retrieving browser information.

Objective

To learn how Selenium WebDriver starts and controls a Chrome browser using Python.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Selenium WebDriver
- Chrome WebDriver
- Browser navigation
- Browser title
- Current URL

Source Code

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
import time
import os

# Create Chrome options
options = Options()

# Start Chrome browser
driver = webdriver.Chrome(options=options)

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open a website
driver.get("https://www.google.com")

# Maximize the browser window
driver.maximize_window()

print("Chrome browser opened successfully.")
print("Current page title:", driver.title)
print("Current URL:", driver.current_url)

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_1.png")
print("Screenshot saved successfully.")

# Keep the browser open for 5 seconds
time.sleep(5)
```

```
# Close the browser
driver.quit()

print("Browser closed successfully.")
```

Procedure

1. Open VS Code and open the Module 1 lab folder.
2. Open the corresponding experiment Python file.
3. Run the program using the Python command in the terminal.
4. Observe the Chrome browser and terminal output.
5. Verify that the screenshot is saved in the screenshots folder.
6. Close the browser after execution.

Output

Screenshot: experiment_1_1.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Chrome browser was successfully launched and controlled using Selenium WebDriver.

Conclusion

The basic working of Selenium WebDriver was successfully demonstrated.

Experiment 1.2 — Selenium WebDriver Introduction and Installation

Problem Statement

To start Chrome using Selenium WebDriver and display browser details such as browser name, version, page title, and URL.

Objective

To understand Selenium WebDriver initialization and browser capabilities.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- WebDriver
- Chrome Options
- Browser capabilities
- Browser version
- Page title
- Current URL

Source Code

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
import os

# Create Chrome options
options = Options()

# Start Chrome using Selenium WebDriver
driver = webdriver.Chrome(options=options)

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium website
driver.get("https://www.selenium.dev/")

# Maximize browser
driver.maximize_window()

print("Selenium WebDriver started successfully.")
print("Browser:", driver.capabilities["browserName"])
print("Browser Version:", driver.capabilities["browserVersion"])
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_2.png")
print("Screenshot saved successfully.")
```

```
# Keep browser open for 5 seconds
import time
time.sleep(5)

# Close browser
driver.quit()

print("WebDriver session closed successfully.")
```

Procedure

7. Open VS Code and open the Module 1 lab folder.
8. Open the corresponding experiment Python file.
9. Run the program using the Python command in the terminal.
10. Observe the Chrome browser and terminal output.
11. Verify that the screenshot is saved in the screenshots folder.
12. Close the browser after execution.

Output

Screenshot: experiment_1_2.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Selenium WebDriver successfully started Chrome and retrieved browser information.

Conclusion

The basic configuration and capabilities of Selenium WebDriver were successfully demonstrated.

Experiment 1.3 — Locators and Object Identification

Problem Statement

To identify web elements using different Selenium locators.

Objective

To understand how Selenium identifies web elements using ID, Name, and Link Text.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- ID locator
- Name locator
- Link Text locator
- `find_element()`
- `send_keys()`

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium practice website
driver.get("https://www.selenium.dev/selenium/web/web-form.html")

# Maximize browser
driver.maximize_window()

print("Website opened successfully.")

# 1. Locate the text box using ID
text_box = driver.find_element(By.ID, "my-text-id")
text_box.send_keys("Selenium")

print("ID locator used successfully.")

# 2. Locate the password field using Name
password_box = driver.find_element(By.NAME, "my-password")
password_box.send_keys("12345")

print("Name locator used successfully.")

# 3. Locate the link using Link Text
```

```
link = driver.find_element(By.LINK_TEXT, "Return to index")
print("Link Text locator found:", link.text)

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_3.png")
print("Screenshot saved successfully.")

# Wait so that the result can be seen
time.sleep(5)

# Close browser
driver.quit()

print("Experiment 3 completed successfully.")
```

Procedure

13. Open VS Code and open the Module 1 lab folder.
14. Open the corresponding experiment Python file.
15. Run the program using the Python command in the terminal.
16. Observe the Chrome browser and terminal output.
17. Verify that the screenshot is saved in the screenshots folder.
18. Close the browser after execution.

Output

Screenshot: experiment_1_3.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

The required web elements were successfully located using different locator strategies.

Conclusion

ID, Name, and Link Text locators were successfully demonstrated.

Experiment 1.4 — Handling Different Controls on Web Page

Problem Statement

To automate different controls available on a web page.

Objective

To handle text boxes, password fields, text areas, dropdowns, checkboxes, radio buttons, and submit buttons.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Text box
- Password field
- Text area
- Dropdown
- Checkbox
- Radio button
- CSS Selector
- Select class

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium Web Form
driver.get("https://www.selenium.dev/selenium/web/web-form.html")

# Maximize browser
driver.maximize_window()

print("Website opened successfully.")

# 1. Handle Text Box
text_box = driver.find_element(By.ID, "my-text-id")
text_box.send_keys("Selenium WebDriver")
print("Text box handled successfully.")

# 2. Handle Password Field
password_box = driver.find_element(By.NAME, "my-password")
password_box.send_keys("12345")
```

```

print("Password field handled successfully.")

# 3. Handle Text Area
text_area = driver.find_element(By.NAME, "my-textarea")
text_area.send_keys("This is Selenium WebDriver practice.")
print("Text area handled successfully.")

# 4. Handle Dropdown
dropdown = Select(driver.find_element(By.NAME, "my-select"))
dropdown.select_by_visible_text("Two")
print("Dropdown handled successfully.")

# 5. Handle Checkbox
checkbox = driver.find_element(By.ID, "my-check-1")

if not checkbox.is_selected():
    checkbox.click()

print("Checkbox handled successfully.")

# 6. Handle Radio Button
radio_button = driver.find_element(By.ID, "my-radio-1")

if not radio_button.is_selected():
    radio_button.click()

print("Radio button handled successfully.")

# Keep browser open
time.sleep(5)

# 7. Handle Submit Button
submit_button = driver.find_element(By.CSS_SELECTOR, "button")
submit_button.click()

print("Submit button clicked successfully.")

# Wait for result
time.sleep(3)

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_4.png")
print("Screenshot saved successfully.")

# Close browser
driver.quit()

print("Experiment 4 completed successfully.")

```

Procedure

19. Open VS Code and open the Module 1 lab folder.
20. Open the corresponding experiment Python file.
21. Run the program using the Python command in the terminal.
22. Observe the Chrome browser and terminal output.
23. Verify that the screenshot is saved in the screenshots folder.
24. Close the browser after execution.

Output

Screenshot: experiment_1_4.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Different web controls were successfully identified and handled using Selenium.

Conclusion

Selenium can automate multiple types of HTML controls on a web page.

Experiment 1.5 — Handling Different Controls — Advanced

Problem Statement

To automate advanced web controls and perform keyboard interactions.

Objective

To understand keyboard actions and controls such as sliders, color pickers, and date pickers.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Keyboard actions
- Keys
- Range slider
- Color picker
- Date picker
- Text field manipulation

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium Web Form
driver.get("https://www.selenium.dev/selenium/web/web-form.html")

# Maximize browser
driver.maximize_window()

print("Website opened successfully.")

# 1. Handle Text Box
text_box = driver.find_element(By.ID, "my-text-id")
text_box.send_keys("Advanced Selenium")

print("Text box handled successfully.")

# Keyboard action
text_box.send_keys(Keys.CONTROL, "a")
text_box.send_keys("Selenium Advanced Controls")

print("Keyboard action performed successfully.")
```

```

# 2. Handle Range Slider
slider = driver.find_element(By.NAME, "my-range")
slider.send_keys(Keys.RIGHT)
slider.send_keys(Keys.RIGHT)

print("Range slider handled successfully.")

# 3. Handle Color Picker
color_picker = driver.find_element(By.NAME, "my-colors")

print("Color picker found successfully.")

# 4. Handle Date Picker
date_picker = driver.find_element(By.NAME, "my-date")
date_picker.send_keys("09/26/2026")

print("Date picker handled successfully.")

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_5.png")
print("Screenshot saved successfully.")

# Keep browser open so the controls can be seen
time.sleep(5)

# Close browser
driver.quit()

print("Experiment 5 completed successfully.")

```

Procedure

25. Open VS Code and open the Module 1 lab folder.
26. Open the corresponding experiment Python file.
27. Run the program using the Python command in the terminal.
28. Observe the Chrome browser and terminal output.
29. Verify that the screenshot is saved in the screenshots folder.
30. Close the browser after execution.

Output

Screenshot: experiment_1_5.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Advanced web controls and keyboard actions were successfully handled.

Conclusion

Selenium provides support for automating advanced controls and keyboard interactions.

Experiment 1.6 — Handling Dynamic Elements

Problem Statement

To identify an element that is dynamically created on a web page.

Objective

To understand how Selenium handles dynamic elements using explicit waits.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Dynamic elements
- WebDriverWait
- Expected Conditions
- presence_of_element_located
- JavaScript execution

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium dynamic page
driver.get("https://www.selenium.dev/selenium/web/dynamic.html")

# Maximize browser
driver.maximize_window()

print("Dynamic elements page opened successfully.")

# Click the button that creates a dynamic element
button = driver.find_element(By.ID, "adder")
button.click()

print("Dynamic element creation button clicked.")

# Wait until the dynamic element appears
wait = WebDriverWait(driver, 10)

dynamic_element = wait.until(
    EC.presence_of_element_located(
```

```

        (By.ID, "box0")
    )
)

print("Dynamic element located successfully.")
print("Dynamic element ID:", dynamic_element.get_attribute("id"))

# Highlight the dynamic element
driver.execute_script(
    "arguments[0].style.border='3px solid red'",
    dynamic_element
)

print("Dynamic element highlighted successfully.")

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_6.png")
print("Screenshot saved successfully.")

# Keep browser open for screenshot
time.sleep(5)

# Close browser
driver.quit()

print("Experiment 6 completed successfully.")

```

Procedure

31. Open VS Code and open the Module 1 lab folder.
32. Open the corresponding experiment Python file.
33. Run the program using the Python command in the terminal.
34. Observe the Chrome browser and terminal output.
35. Verify that the screenshot is saved in the screenshots folder.
36. Close the browser after execution.

Output

Screenshot: experiment_1_6.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

The dynamically created element was successfully located and highlighted.

Conclusion

Explicit waits can be used to handle dynamically appearing web elements.

Experiment 1.7 — Synchronization / Wait Type

Problem Statement

To understand synchronization in Selenium using implicit and explicit waits.

Objective

To demonstrate how waits help Selenium interact with elements that may not be immediately available.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Implicit Wait
- Explicit Wait
- WebDriverWait
- Expected Conditions
- Dynamic elements

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Implicit Wait
driver.implicitly_wait(10)

# Open Selenium dynamic page
driver.get("https://www.selenium.dev/selenium/web/dynamic.html")

print("Website opened successfully.")

# Click the button to create a dynamic element
button = driver.find_element(By.ID, "adder")
button.click()

print("Dynamic element creation button clicked.")

# Explicit Wait
wait = WebDriverWait(driver, 10)

dynamic_element = wait.until(
    EC.presence_of_element_located(
```

```

        (By.ID, "box0")
    )
)

print("Explicit wait completed successfully.")
print("Dynamic element found:", dynamic_element.get_attribute("id"))

# Highlight the element
driver.execute_script(
    "arguments[0].style.border='3px solid red'",
    dynamic_element
)

print("Dynamic element highlighted successfully.")

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_7.png")
print("Screenshot saved successfully.")

# Keep browser open for screenshot
time.sleep(5)

# Close browser
driver.quit()

print("Experiment 7 completed successfully.")

```

Procedure

37. Open VS Code and open the Module 1 lab folder.
38. Open the corresponding experiment Python file.
39. Run the program using the Python command in the terminal.
40. Observe the Chrome browser and terminal output.
41. Verify that the screenshot is saved in the screenshots folder.
42. Close the browser after execution.

Output

Screenshot: experiment_1_7.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Implicit and explicit waits were successfully demonstrated.

Conclusion

Synchronization techniques help Selenium wait for web elements before performing actions.

Experiment 1.8 — Take Screenshots & Exception Handling

Problem Statement

To capture screenshots during Selenium execution and handle exceptions when an element cannot be found.

Objective

To understand screenshot capturing and exception handling using Selenium.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- Screenshots
- try
- finally
- NoSuchElementException
- Exception handling
- save_screenshot()

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.common.exceptions import NoSuchElementException
import time
import os

# Start Chrome
driver = webdriver.Chrome()

try:
    # Open Selenium Web Form
    driver.get("https://www.selenium.dev/selenium/web/web-form.html")

    print("Website opened successfully.")

    # Create screenshots folder if it does not exist
    os.makedirs("screenshots", exist_ok=True)

    # Take screenshot of the webpage
    screenshot_path = "screenshots/experiment_1_8.png"
    driver.save_screenshot(screenshot_path)

    print("Screenshot taken successfully.")
    print("Screenshot saved at:", screenshot_path)

    # Try to find a valid element
    text_box = driver.find_element(By.ID, "my-text-id")
    text_box.send_keys("Exception Handling")

    print("Valid element found successfully.")
```

```
# Try to find an element that does not exist
try:
    driver.find_element(By.ID, "element-that-does-not-exist")

except NoSuchElementException:
    print("Exception handled: Element was not found.")

# Keep browser open
time.sleep(5)

finally:
    # Close browser
    driver.quit()

print("Browser closed successfully.")
print("Experiment 8 completed successfully.")
```

Procedure

43. Open VS Code and open the Module 1 lab folder.
44. Open the corresponding experiment Python file.
45. Run the program using the Python command in the terminal.
46. Observe the Chrome browser and terminal output.
47. Verify that the screenshot is saved in the screenshots folder.
48. Close the browser after execution.

Output

Screenshot: experiment_1_8.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

The screenshot was successfully captured and the NoSuchElementException was handled successfully.

Conclusion

Selenium screenshots and Python exception handling can be used together to make automation more reliable.

Experiment 1.9 — Advanced Interactions

Problem Statement

To perform advanced mouse interactions using Selenium ActionChains.

Objective

To automate mouse hover, double-click, right-click, and drag-and-drop operations.

Tools / Software Used

- Python
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used

- ActionChains
- Mouse Hover
- Double Click
- Right Click
- Drag and Drop

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
import time
import os

# Start Chrome
driver = webdriver.Chrome()

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Open Selenium mouse interaction page
driver.get("https://www.selenium.dev/selenium/web/mouse_interaction.html")

print("Advanced interactions page opened successfully.")

# Create ActionChains object
actions = ActionChains(driver)

# 1. Mouse Hover
hover_element = driver.find_element(By.ID, "hover")
actions.move_to_element(hover_element).perform()

print("Mouse hover performed successfully.")

time.sleep(2)

# 2. Double Click
clickable_element = driver.find_element(By.ID, "clickable")
actions.double_click(clickable_element).perform()
```

```
print("Double-click performed successfully.")

time.sleep(2)

# 3. Right Click
right_click_element = driver.find_element(By.ID, "clickable")
actions.context_click(right_click_element).perform()

print("Right-click performed successfully.")

time.sleep(2)

# 4. Drag and Drop
source = driver.find_element(By.ID, "draggable")
target = driver.find_element(By.ID, "droppable")

actions.drag_and_drop(source, target).perform()

print("Drag and drop performed successfully.")

# Take screenshot
driver.save_screenshot("screenshots/experiment_1_9.png")
print("Screenshot saved successfully.")

# Keep browser open for screenshot
time.sleep(5)

# Close browser
driver.quit()

print("Experiment 9 completed successfully.")
```

Procedure

49. Open VS Code and open the Module 1 lab folder.
50. Open the corresponding experiment Python file.
51. Run the program using the Python command in the terminal.
52. Observe the Chrome browser and terminal output.
53. Verify that the screenshot is saved in the screenshots folder.
54. Close the browser after execution.

Output

Screenshot: experiment_1_9.png

[INSERT ACTUAL SCREENSHOT HERE]

Result

Advanced mouse interactions were performed using Selenium ActionChains.

Conclusion

Selenium ActionChains can be used to automate complex mouse interactions.

Overall Conclusion

The nine experiments of Module 1 – Automation with Selenium were completed using Python and Selenium WebDriver. The experiments covered Selenium WebDriver basics, element locators, web controls, advanced controls, dynamic elements, synchronization, screenshots, exception handling, and advanced mouse interactions. Through these experiments, Selenium WebDriver automation concepts were practically demonstrated using Google Chrome.

Final Screenshot Checklist

- ☐ experiment_1_1.png
- ☐ experiment_1_2.png
- ☐ experiment_1_3.png
- ☐ experiment_1_4.png
- ☐ experiment_1_5.png
- ☐ experiment_1_6.png
- ☐ experiment_1_7.png
- ☐ experiment_1_8.png
- ☐ experiment_1_9.png